

41 - Application Log Viewer with In-Memory Ring Buffer

What It Is

This feature combines five building blocks to give admins a live window into what the server is doing -- without ever needing SSH access or reading raw log files on disk.

1. In-Memory Ring Buffer (`collections.deque(maxlen=N)`)

A **ring buffer** (also called a circular buffer) is a fixed-size container that works like a conveyor belt. Imagine a belt with exactly 500 slots. Every new log message goes onto the next slot. Once all 500 slots are full, the belt loops back to the beginning and the oldest message gets pushed off the end to make room for the newest one.

Python gives us this for free with `collections.deque(maxlen=N)`. A `deque` (pronounced "deck") is a **double-ended queue** -- a list-like structure optimized for fast appends and pops from both ends. When you pass `maxlen=500`, Python enforces the size limit automatically: every `.append()` that would exceed 500 items silently discards the oldest item on the opposite end. No manual bookkeeping needed.

Why not a plain list? A list has no built-in size cap -- you would need to manually `pop(0)` (which is slow, $O(n)$, because it shifts every element) or track indices yourself. `deque` does constant-time $O(1)$ appends and evictions.

2. Loguru Custom Sinks

Loguru is the logging library used throughout this project. By default, log messages go to the console (stdout). But loguru lets you add **sinks** -- destinations where log messages should also be sent.

A sink can be a file path (loguru writes to the file), or it can be any **callable** (a function). When you pass a function as a sink, loguru calls that function with every log message. The function receives a `message` object whose `.record` attribute contains structured data: timestamp, level (DEBUG/INFO/WARNING/ERROR), module name, function name, line number, and the message text.

This is the key insight: by writing a small function that extracts these fields into a dictionary and appends it to our ring buffer, we turn the logging system into a structured data source that any other part of the application can query.

3. Thread Safety with `threading.Lock`

A web server like FastAPI handles multiple requests concurrently. If two requests trigger log messages at the same instant, both would try to append to the same deque simultaneously. This is a **race condition** -- concurrent writes to shared mutable state can corrupt data or cause crashes.

A `threading.Lock` is like a single-key bathroom lock. Only one thread can hold the lock at a time. Any other thread that tries to acquire it has to wait until the first thread releases it. By wrapping every read and write to the deque inside `with _lock:`, we guarantee that only one thread touches the buffer at any given moment.

The `with` statement is critical here -- it ensures the lock is released even if an exception occurs inside the block (it calls `_lock.release()` automatically in the `finally` clause).

4. Admin-Only Log Endpoint

The logs are served via a REST API endpoint (`GET /admin/logs`) that only authenticated administrators can access. The endpoint accepts three optional query parameters:

- `limit` -- how many entries to return (1-500, default 100)
- `level` -- filter to a specific log level (e.g., only ERROR entries)
- `search` -- text search within log messages (case-insensitive substring match)

The `require_admin` dependency ensures that only requests with a valid admin JWT token can access the logs. This prevents regular users (or attackers) from seeing internal server details.

5. React Live Log Viewer

The admin panel includes a dedicated Logs page that presents the log entries in a terminal-like monospace display. It:

- **Polls** the API every 5 seconds (configurable via auto-refresh toggle)
- **Color-codes** log levels: gray for DEBUG, blue for INFO, amber for WARNING, red for ERROR
- **Provides** a level dropdown filter and a text search input
- **Auto-scrolls** to the bottom so the newest entries are always visible
- **Shows** an entry count and a manual refresh button

Real-Life Analogy

Think of a **security camera system with a fixed-size DVR**.

The cameras record continuously, 24/7. But the DVR hard drive only holds the last 72 hours of footage. When hour 73 arrives, the oldest hour (hour 1) gets overwritten automatically. The DVR never runs out of space, and nobody has to manually delete old footage.

Now picture the **security monitor room**. Guards sit in front of screens watching the live feed. They can:

- **Filter by camera** (like filtering by log level) -- "Show me only the parking lot camera" is like "Show me only ERROR entries."
- **Search for events** (like text search) -- "Show me footage where motion was detected near the back door" is like searching for "embedding" in the logs.
- **Only authorized guards get access** (like `require_admin`) -- you need a badge (JWT token) to enter the monitor room.

The ring buffer is the DVR. The loguru sink is the cable connecting cameras to the DVR. The lock is the single remote control -- only one guard can rewind/search at a time. The REST endpoint is the door to the monitor room. The React viewer is the bank of screens on the wall.

Why We Used It Here

This feature addresses a practical need: **admins need to see what the server is doing without SSH access**.

In production, the admin user might be a project manager or client representative who cannot (and should not) log into the server via a terminal. But they still need to:

- Confirm that search queries are being processed correctly
- See which embedding provider is handling requests
- Watch RRF fusion scores in real time
- Spot errors (failed API calls, database timeouts) as they happen
- Verify that the ETL pipeline log messages appear during a run

The ring buffer approach has three important properties:

1. **Zero disk I/O** -- everything lives in memory, so logging to the buffer adds negligible latency to request handling.

2. **Bounded memory** -- the deque never grows beyond 500 entries. Even if each entry is 1 KB, that is 500 KB maximum. No risk of filling the disk or consuming unbounded RAM.
 3. **Security** -- the endpoint is protected by `require_admin` (JWT auth), so only authenticated admins can read the logs. Internal details like module names, function names, and line numbers are never exposed to unauthenticated users.
-

Code Walkthrough

File 1: `search_api/services/log_buffer.py` -- The Ring Buffer and Sink

This is the core module. It defines the buffer, the loguru sink function, and the query function.

```
"""
In-memory ring buffer for recent log entries.

Loguru sink that captures the last N log messages so they can be
served to the admin UI via the /admin/logs endpoint.
"""

import threading
from collections import deque

# Maximum number of log entries to keep in memory.
# 500 is a good balance: enough history for debugging,
# small enough to use negligible RAM (~500 KB worst case).
_MAX_ENTRIES = 500

# The ring buffer itself. maxlen=500 means Python will automatically
# discard the oldest entry when a 501st is appended.
_buffer: deque[dict] = deque(maxlen=_MAX_ENTRIES)

# A threading lock to prevent concurrent read/write corruption.
# FastAPI serves requests in multiple threads, so we need this.
_lock = threading.Lock()

def log_sink(message):
    """Loguru sink function -- called for every log message.

    `message` is a loguru Message object. Its `.record` attribute
    contains structured metadata about the log entry.
    We extract the fields we care about into a plain dict
    and append it to the ring buffer.
    """
    record = message.record
    entry = {
        # Format timestamp as "2026-09-10 14:23:45.123" (trim microseconds to ms)
```

```

        "timestamp": record["time"].strftime("%Y-%m-%d %H:%M:%S.%f")[:-3],
        "level": record["level"].name,          # "DEBUG", "INFO", "WARNING", "ERROR"
        "module": record["module"],             # e.g. "rrf_fusion"
        "function": record["function"],         # e.g. "fuse_results"
        "line": record["line"],                 # e.g. 42
        "message": record["message"],           # the actual log text
    }
    # Acquire the lock before writing to the shared buffer.
    # `with` ensures the lock is released even if an exception occurs.
    with _lock:
        _buffer.append(entry)

def get_recent_logs(
    limit: int = 100,
    level: str | None = None,
    search: str | None = None,
) -> list[dict]:
    """Return recent log entries, optionally filtered.

    Steps:
    1. Snapshot the buffer under the lock (fast -- just copies pointers).
    2. Filter by level if requested.
    3. Filter by text search if requested.
    4. Return the last `limit` entries from the filtered result.
    """
    # Take a snapshot while holding the lock.
    # list(_buffer) copies the deque contents into a plain list,
    # so we release the lock quickly and filter without holding it.
    with _lock:
        entries = list(_buffer)

    # Level filter: compare uppercase to normalize "error" -> "ERROR"
    if level:
        level_upper = level.upper()
        entries = [e for e in entries if e["level"] == level_upper]

    # Text search: case-insensitive substring match in the message field
    if search:
        search_lower = search.lower()
        entries = [e for e in entries if search_lower in e["message"].lower()]

    # Return only the most recent `limit` entries (tail of the list)
    return entries[-limit:]

```

Key design decisions:

- **Snapshot then filter:** We copy the buffer under the lock (`list(_buffer)`) and release the lock immediately. Filtering happens outside the lock, so a slow search query does not block other threads from writing logs.
- **Tail slice** (`entries[-limit:]`): After filtering, we return the most recent entries, not the first N. This ensures the viewer always sees the latest activity.
- **Module-level globals:** `_buffer` and `_lock` are module-level singletons. Every import of `log_buffer` gets the same objects -- no class instantiation needed.

File 2: `search_api/routers/logs.py` -- The Admin Endpoint

This file defines the FastAPI route that serves log entries to the admin UI.

```
"""
GET /admin/logs -- Recent application logs (admin only).
"""

from fastapi import APIRouter, Depends, Query

from search_api.routers.auth import require_admin
from search_api.services.log_buffer import get_recent_logs

router = APIRouter()

@router.get("/admin/logs", dependencies=[Depends(require_admin)])
def get_logs(
    # Query parameters with validation:
    # - limit: 1-500, defaults to 100
    # - level: optional string, e.g. "ERROR"
    # - search: optional string, max 200 chars to prevent abuse
    limit: int = Query(100, ge=1, le=500),
    level: str | None = Query(None, description="Filter by level: DEBUG, INFO, WARNING, ERROR"),
    search: str | None = Query(None, max_length=200, description="Filter by text in message"),
):
    # Delegate entirely to the service layer.
    # The endpoint is thin on purpose -- all logic lives in log_buffer.py.
    return get_recent_logs(limit=limit, level=level, search=search)
```

Key points:

- **`dependencies=[Depends(require_admin)]`** : This is FastAPI's dependency injection. Before the handler runs, FastAPI calls `require_admin`, which checks the JWT token in the request headers. If the token is missing or invalid, it raises a 401/403 error and the handler never executes.
- **Query() with validation**: `ge=1, le=500` ensures the limit is between 1 and 500. `max_length=200` on search prevents a caller from sending a megabyte-long search string.
- **Thin endpoint**: The route handler does nothing except call `get_recent_logs()`. This keeps the router file focused on HTTP concerns (parameter validation, auth) while all log logic stays in the service layer.

File 3: Sink Registration in `search_api/main.py` (line 122)

This single line connects the logging system to the ring buffer:

```
logger.add(log_sink, level="DEBUG", format="{message}")
```

What each argument does:

- `log_sink` -- the function from `log_buffer.py`. Loguru will call this function for every log message that passes the level filter.
- `level="DEBUG"` -- capture all log levels (DEBUG is the lowest). This means INFO, WARNING, ERROR, and CRITICAL messages are all captured too.
- `format="{message}"` -- tells loguru what string to pass as `str(message)`. Since our sink reads `message.record` (the structured data), the format string does not matter much here, but `"{message}"` keeps it clean.

This line runs once at application startup (module load time), before any requests arrive. From that point on, every `logger.info(...)`, `logger.error(...)`, etc. throughout the entire codebase automatically flows into the ring buffer.

File 4: `admin-ui/src/pages/Logs.jsx` -- The React Log Viewer

This is the frontend component that renders the log viewer in the admin panel.

```
import { useState, useEffect, useRef } from "react";
import { getLogs } from "../api.js";

// Color map for log levels -- makes it easy to spot errors at a glance.
const LEVEL_COLORS = {
  DEBUG: "#9ca3af",    // gray -- low importance, slightly transparent
  INFO: "#3b82f6",     // blue -- normal operations
  WARNING: "#f59e0b",  // amber -- something to watch
  ERROR: "#ef4444",    // red -- needs attention
  CRITICAL: "#dc2626", // dark red -- something is very wrong
};

export default function Logs() {
  // State: the log entries array, filter values, and UI toggles
  const [entries, setEntries] = useState([]);
  const [level, setLevel] = useState(""); // "" means "all levels"
  const [search, setSearch] = useState("");
  const [autoRefresh, setAutoRefresh] = useState(true);
  const [loading, setLoading] = useState(true);
  const bottomRef = useRef(null); // ref to scroll target at the bottom

  // Fetch logs from the API, applying current filters
  async function fetchLogs() {
    try {
      const data = await getLogs(300, level || null, search || null);
      setEntries(data);
    } catch { /* auth redirect handled by api.js */ }
    setLoading(false);
  }

  // Effect: fetch on mount, then set up polling interval if auto-refresh is on
  useEffect(() => {
    fetchLogs();
    if (!autoRefresh) return; // no interval if auto-refresh is off
    const interval = setInterval(fetchLogs, 5000); // poll every 5 seconds
  }, [autoRefresh]);
}
```

```

    return () => clearInterval(interval); // cleanup on unmount or re-render
  }, [level, search, autoRefresh]); // re-run when filters or toggle change

// Effect: auto-scroll to bottom when new entries arrive
useEffect(() => {
  if (autoRefresh && bottomRef.current) {
    bottomRef.current.scrollToView({ behavior: "smooth" });
  }
}, [entries]);

if (loading) return (
  <div className="card">
    <p style={{ color: "var(--text-muted)" }}>Loading logs...</p>
  </div>
);

return (
  <>
    {/* Filter toolbar */}
    <div className="card" style={{
      display: "flex", gap: "0.75rem",
      alignItems: "center", flexWrap: "wrap"
    }}>
      {/* Level dropdown filter */}
      <select value={level} onChange={(e) => setLevel(e.target.value)}
        style={{ /* styling omitted for brevity */ }}>
        <option value="">All levels</option>
        <option value="DEBUG">DEBUG</option>
        <option value="INFO">INFO</option>
        <option value="WARNING">WARNING</option>
        <option value="ERROR">ERROR</option>
      </select>

      {/* Text search input */}
      <input type="text" placeholder="Search logs..."
        value={search} onChange={(e) => setSearch(e.target.value)}
        style={{ /* styling omitted for brevity */ }} />

      {/* Auto-refresh toggle */}
      <label>
        <input type="checkbox" checked={autoRefresh}
          onChange={(e) => setAutoRefresh(e.target.checked)} />
        Auto-refresh (5s)
      </label>

      {/* Manual refresh button */}
      <button onClick={fetchLogs} className="chip">Refresh</button>

      {/* Entry count */}
      <span>{entries.length} entries</span>
    </div>

    {/* Log entries display -- terminal-like monospace */}
    <div className="card" style={{
      fontFamily: "'JetBrains Mono', 'Fira Code', monospace",
      fontSize: "0.72rem", lineHeight: "1.6",
      maxHeight: "65vh", overflowY: "auto",

```

```

    whiteSpace: "pre-wrap", wordBreak: "break-word",
  }}>
  {entries.length === 0 ? (
    <p>No log entries{level ? ` at ${level} level` : ""}
    {search ? ` matching "${search}"` : ""}.</p>
  ) : (
    entries.map((e, i) => (
      <div key={i} style={{
        padding: "0.15rem 0",
        borderBottom: "1px solid var(--border)",
        opacity: e.level === "DEBUG" ? 0.6 : 1, // dim DEBUG entries
      }}>
        {/* Timestamp (time portion only) */}
        <span style={{ color: "var(--text-muted)" }}>
          {e.timestamp.split(" ")[1]}
        </span>{" "}
        {/* Level badge -- color-coded, fixed width for alignment */}
        <span style={{
          color: LEVEL_COLORS[e.level] || "var(--text)",
          fontWeight: e.level === "ERROR" || e.level === "WARNING" ? 600 : 400,
          display: "inline-block", width: "5.5ch",
        }}>
          {e.level.padEnd(5)}
        </span>{" "}
        {/* Source location: module:function:line */}
        <span style={{ color: "var(--primary)", opacity: 0.7 }}>
          {e.module}:{e.function}:{e.line}
        </span>{" "}
        {/* The actual log message */}
        <span style={{
          color: e.level === "ERROR" ? LEVEL_COLORS.ERROR : "var(--text)"
        }}>
          {e.message}
        </span>
      </div>
    ))
  )}
  {/* Invisible element at the bottom for auto-scroll targeting */}
  <div ref={bottomRef} />
</div>
</>
);
}

```

Key React patterns used:

- **useEffect with cleanup:** The polling interval is set up inside `useEffect` and cleaned up by returning `() => clearInterval(interval)`. This prevents memory leaks when the component unmounts or when filters change.
- **useRef for scroll targeting:** `bottomRef` is attached to an invisible `<div>` at the end of the log list. Calling `scrollIntoView()` on it scrolls the container to the bottom.

- **Controlled inputs:** The level dropdown and search input are controlled components -- their values come from React state, and `onChange` updates the state. This means the filter values are always in sync with what the user sees.

File 5: `getLogs()` from `admin-ui/src/api.js` (lines 123-129)

This is the API client function that the Logs component calls:

```
async function getLogs(limit = 200, level = null, search = null) {  
  // Build query string from the provided parameters  
  const params = new URLSearchParams({ limit: String(limit) });  
  if (level) params.set("level", level);    // only include if not null  
  if (search) params.set("search", search); // only include if not null  
  
  // apiFetch is a wrapper that adds the JWT Authorization header  
  // and handles 401 redirects to the login page  
  const resp = await apiFetch(`/admin/logs?${params}`);  
  return resp.json();  
}
```

This function builds the query string dynamically, only including `level` and `search` parameters when they have values. `apiFetch` is a project-wide wrapper around `fetch()` that automatically attaches the JWT token from local storage and redirects to the login page if the server returns 401 Unauthorized.

Data Flow Summary

Here is the complete journey of a log message from creation to display:

```
1. Code calls logger.info("RRF fusion: 42 results")  
   |  
2. Loguru routes message to all registered sinks  
   |  
3. log_sink() extracts structured fields from message.record  
   |  
4. Acquires _lock, appends dict to _buffer (deque), releases _lock  
   |  
5. Admin opens Logs page in browser  
   |  
6. React useEffect triggers fetchLogs() immediately + every 5 seconds  
   |  
7. getLogs() calls GET /admin/logs?limit=300 with JWT header  
   |  
8. FastAPI runs require_admin dependency (validates JWT)  
   |  
9. get_recent_logs() snapshots _buffer under _lock, filters, returns tail  
   |  
10. React renders entries with level colors and auto-scrolls to bottom
```

How to Explain This in an Interview / to a Teammate

"We built an in-browser log viewer for admins using a ring buffer pattern. On the backend, we use Python's `collections.deque(maxlen=500)` as a fixed-size in-memory buffer -- it automatically evicts the oldest entry when full, so memory usage is bounded and there is zero disk I/O. We plugged it into our logging system using a loguru custom sink: a plain function that receives every log message, extracts structured fields (timestamp, level, module, message), and appends them to the buffer behind a `threading.Lock` for thread safety. The buffer is exposed via a FastAPI endpoint protected by JWT admin auth, with optional level and text search filtering. On the frontend, a React component polls that endpoint every five seconds, renders entries in a color-coded monospace view, and auto-scrolls to the latest entry. The whole feature gives admins real-time server visibility without SSH access, and the ring buffer guarantees we never consume more than about 500 KB of RAM regardless of traffic volume."